

Atividade 05 - Expressões Constantes 1 - Análise Sintática

Grupo: 20230013339 — João Marcos Cunha Santos

Compilador de expressões aritméticas com duas etapas:

1. **Análise léxica** (`src/lexical.rs`) — identifica e classifica tokens: números, operadores (+, -, *, /), parênteses e espaços.
2. **Análise sintática** (`src/syntax.rs`) — monta a **árvore de expressão (AST)** a partir da lista de tokens.

O fluxo em `src/main.rs` lê o arquivo de entrada, tokeniza o conteúdo, constrói a árvore e imprime o resultado no terminal.

Árvore de sintaxe

A AST é representada pelo enum `Expression`:

- `NumberLiteral(i32)` — folha com um número inteiro.
- `BinOperation { left_value, operator, right_value }` — operação binária com filhos em `Box<Expression>`.

Os operadores são modelados pelo enum `Operator` (`Sum`, `Sub`, `Div`, `Mul`).

A gramática reconhecida segue o padrão (`expressão operador expressão`), com parênteses aninhados. Espaços são ignorados durante a análise.

Exemplo de saída para `(3 + (4 + (11 + 7)))`:

result tree:

└─ BinOp(Sum)

└─ Number(3)

└─ BinOp(Sum)

└─ Number(4)

└─ BinOp(Sum)

└─ Number(11)

└─ Number(7)

Observações

- A gramática é limitada, não suporta expressões sem parênteses como $(3 + 4) * 5$ ou $3 + 4 * 5$.

Como executar

Requisito: Rust instalado.

Executar o compilador sobre um arquivo de entrada:

```
cargo run <CAMINHO_DO_ARQUIVO>
```

Testes de sucesso

```
cargo run test/success/source_code_1
```

```
cargo run test/success/source_code_2
```

```
cargo run test/success/source_code_3
```

Testes de erro

```
cargo run test/error/source_code_1
```

```
cargo run test/error/source_code_2
```

```
cargo run test/error/source_code_3
```

test/success/source_code_1

Entrada: $(3 + (4 + (11 + 7)))$

result tree:

└─ BinOp(Sum)

├─ Number(3)

└─ BinOp(Sum)

├─ Number(4)

└─ BinOp(Sum)

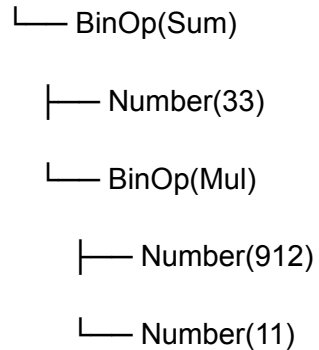
├─ Number(11)

└─ Number(7)

test/success/source_code_2

Entrada: $(33 + (912 * 11))$

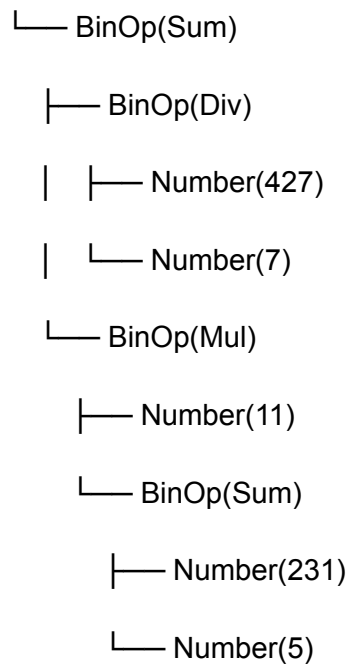
result tree:



test/success/source_code_3

Entrada: $((427 / 7) + (11 * (231 + 5)))$

result tree:



test/error/source_code_1

Entrada: $54 + x / 98 (4 + 2)$ — caractere inválido na análise léxica.

Error lexical: invalid character 'x' at 1:6

test/error/source_code_2

Entrada: $123a + 23$ — caractere inválido na análise léxica.

Error lexical: invalid character 'a' at 1:4

test/error/source_code_3

Entrada: arquivo vazio — falha na análise sintática por falta de tokens.

Error syntax: Miss token to complete expression